

DEPLOYMENT

Important Note

Deployments are recommended over ReplicaSets.

Reason:

- ReplicaSet has scaling and self-healing.
- But ReplicaSet cannot properly handle:
 - Rollout
 - Rollback
 - Application updates

Deployment solves these problems.

Kubernetes Deployment

Definition

Kubernetes Deployment is a high-level resource object by which you can manage deployment and scaling of applications while maintaining the desired state of the application.

Beginner Friendly Explanation

Deployment is a Kubernetes object used to:

- Deploy applications
- Scale applications
- Update applications
- Rollback applications
- Maintain desired number of PODs automatically

Deployment is one of the most important Kubernetes objects used in real-time projects.

Important Faculty Explanation

You can scale the containers with the help of Kubernetes Deployment up and down depending on incoming traffic.

Meaning:

- More traffic → Increase PODs
- Less traffic → Decrease PODs

This helps:

- Handle users efficiently
 - Save resources
-

Rolling Update Feature

Important Definition

If you have performed any rolling updates with the help of deployment and later find any bugs, then you can perform rollback also.

What is Rolling Update?

Rolling update means:

- Updating application version gradually
- Without downtime

Example:

- Old Version → v1
- New Version → v2

Deployment slowly replaces old PODs with new PODs.

What is Rollback?

Rollback means:

- Returning application to previous stable version.

Example:

- Updated application has bugs.
- Deployment can restore previous working version.

ReplicaSet cannot do this properly.

Important Faculty Line

Kubernetes deployments are deployed with the help of CLI like kubectl.

Meaning:

- We use kubectl commands to manage deployments.
-

Relationship Between Deployment, ReplicaSet and PODs

Earlier

ReplicaSet → Creates PODs

Now

Deployment → Creates ReplicaSet → ReplicaSet Creates PODs

Important Architecture

Deployment



ReplicaSet



PODs

Important Faculty Statement

deployment create rs ---> rs will create pods

Meaning:

- Deployment internally creates ReplicaSet.
- ReplicaSet internally creates PODs.

Major Advantage of Deployment

We can update the application.

This is biggest improvement over ReplicaSet.

First Delete Existing ReplicaSet

Before creating Deployment:

- Delete old ReplicaSet.

Command:

```
kubectrl delete rs ib-rs
```

Verify ReplicaSet Deleted

Command:

```
kubectrl get rs
```

Purpose:

- Check available ReplicaSets.

Verify PODs

Command:

```
kubectl get pods
```

Purpose:

- Verify POD status.

Create Deployment YAML from ReplicaSet YAML

Instead of writing YAML from beginning:

Command:

```
cp replicaset.yml deployment.yml
```

Purpose:

- Copy ReplicaSet YAML into Deployment YAML.

Modify Deployment YAML

Open YAML:

```
vi deployment.yml
```

Important Faculty Instruction

Just change:

kind: Deployment

and

name: ib-deployment

Also Change Image

Faculty Note:

Change image to internetbanking for understanding better.

Deployment YAML File

apiVersion: apps/v1

kind: Deployment

metadata:

name: ib-deployment

labels:

app: bank

spec:

replicas: 3

selector:

matchLabels:

app: bank

template:

metadata:

labels:

app: bank

spec:

containers:

- name: cont1

image: trainerreyaz/ib-image:latest

YAML Explanation

apiVersion

apiVersion: apps/v1

Purpose:

- Uses stable apps API.
-

kind

kind: Deployment

Purpose:

- Creates Deployment object.
-

metadata

metadata:

name: ib-deployment

Purpose:

- Name of deployment.
-

labels

labels:

app: bank

Purpose:

- Group related resources.
-

replicas

replicas: 3

Purpose:

- Maintain 3 PODs.
-

selector

selector:

matchLabels:

app: bank

Purpose:

Select PODs with label:

app=bank

-
-

template

Template defines:

- POD blueprint/template.
-

containers

containers:

- name: cont1

Purpose:

- Container name.
-

image

image: trainerreyaz/ib-image:latest

Purpose:

- Docker image used for container creation.
-

Create Deployment

Command:

```
kubectl create -f deployment.yml
```

Purpose:

- Creates deployment from YAML file.
-

Verify Deployments

Command:

```
kubectl get deploy
```

OR

```
kubectl get deployments
```

Purpose:

- List deployments.
-

Detailed Output

Command:

```
kubectl get deploy -o wide
```

Purpose:

- Shows detailed deployment information.
-

Verify ReplicaSet Created by Deployment

Command:

kubectl get rs

Important Faculty Note

Deploy will create ReplicaSet

Meaning:

- Deployment internally creates ReplicaSet automatically.
-

Describe Deployment

Command:

kubectl describe deploy ib-deployment

Purpose:

- Shows complete deployment details.

Includes:

- Replica count
- Image name
- Labels
- Events
- Strategy
- POD information

Real-Time Flow

Step-by-Step

1. Create Deployment
 2. Deployment creates ReplicaSet
 3. ReplicaSet creates PODs
 4. Deployment manages updates
 5. Deployment handles rollback
-

Why Deployment is Better Than ReplicaSet?

Feature	ReplicaSet	Deployment
Self Healing	Yes	Yes
Scaling	Yes	Yes
Rolling Updates	No Properly	Yes
Rollback	No	Yes
Application Update	Difficult	Easy
Real-Time Usage	Rarely	Mostly Used

Important Interview Question

Question:

Why Deployment is preferred over ReplicaSet?

Answer:

Deployment is preferred because it provides:

- Rolling updates
- Rollback
- Easy application updates
- Better application management
- Self-healing
- Scaling

Deployment internally uses ReplicaSet to create and manage PODs.

Important Interview Question

Question:

What is the relationship between Deployment and ReplicaSet?

Answer:

Deployment creates ReplicaSet
ReplicaSet creates PODs

Deployment acts as higher-level controller.

Important Interview Question

Question:

What is rolling update?

Answer:

Rolling update means:

- Updating application version gradually
- Without downtime

Deployment replaces old PODs with new PODs one by one.

Important Interview Question

Question:

What is rollback?

Answer:

Rollback means:

- Returning application to previous stable version when new version has bugs.
-

Final Quick Revision

Deployment is a high-level Kubernetes object.

Deployment:

- Creates ReplicaSet
- ReplicaSet Creates PODs

Deployment advantages:

- Scaling
- Self Healing
- Rolling Updates
- Rollback
- Application Updates

ReplicaSet cannot properly handle updates,
so Deployment is recommended in real-time projects.

Vi deployment.yml

correct deployment.yml

apiVersion: apps/v1

kind: Deployment

metadata:

name: ib-deployment

labels:

app: bank

spec:

```
replicas: 3

selector:
  matchLabels:
    app: bank

template:
  metadata:
    labels:
      app: bank

spec:
  containers:
    - name: cont1
      image: trainerreyaz/ib-image:latest
```

Copy-Paste Commands

Step 1 — Create Deployment File

vi deployment.yml

Paste the YAML above.

Save file:

ESC → :wq

Step 2 — Create Deployment

kubectl apply -f deployment.yml

Step 3 — Verify Deployment

kubectl get deployments

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE
ib-deployment	3/3	3	3

Step 4 — Check ReplicaSet

kubectl get rs

Step 5 — Check Pods

kubectl get pods

Step 6 — Describe Deployment

kubectl describe deployment ib-deployment

Step 7 — Delete Deployment

kubectl delete -f deployment.yml

or

kubectl delete deployment ib-deployment

Scale Deployment

Increase replicas:

kubectl scale deployment ib-deployment --replicas=5

Update Image

kubectl set image deployment/ib-deployment cont1=nginx

Rollout Status

kubectl rollout status deployment ib-deployment

Important Beginner Understanding

Deployment

A Deployment:

- Manages ReplicaSets
 - Provides rolling updates
 - Handles scaling
 - Automatically recreates failed pods
-

Difference

Resource	Purpose
Pod	Runs container
ReplicaSet	Maintains fixed number of pods
Deployment	Manages ReplicaSets and updates

Mistakes Fixed from Screenshot

Problem	Fixed
Explanation text inside YAML	Removed
Broken indentation	Corrected

Mixed notes with manifest	Separated
Formatting issues	Fixed

Multi-Container POD in Kubernetes

What is Multi-Container POD?

Normally:

- One POD contains one container.

But Kubernetes also supports:

- Multiple containers inside a single POD.

All containers inside the same POD:

- Share same network
 - Share same storage volumes
 - Communicate using localhost
-

Important Concept

One POD can contain multiple containers.

These containers work together to complete one application task.

Real-Time Example

Example:

- One container runs Nginx web server
- Another container runs helper/background process

Both containers work together inside same POD.

Multi-Container POD YAML

apiVersion: v1

kind: Pod

metadata:

name: two-container-pod

labels:

app: multi-container

spec:

containers:

- name: nginx-container

image: nginx

ports:

- containerPort: 80

- name: busybox-container

image: busybox

command: ["sh", "-c", "while true; do echo 'Hello from BusyBox'; sleep 10; done"]

YAML Explanation

apiVersion

apiVersion: v1

Purpose:

- Uses Kubernetes core API version.
-

kind

kind: Pod

Purpose:

- Creates a POD object.
-

metadata

metadata:

name: two-container-pod

Purpose:

- Name of POD.
-

labels

labels:

app: multi-container

Purpose:

- Labels help group and identify resources.
-

spec

spec:

Purpose:

- Contains POD configuration.
-

containers Section

containers:

Purpose:

- Defines containers inside POD.
-

First Container : Nginx

- name: nginx-container
image: nginx

Purpose:

- Creates Nginx web server container.
-

Ports

ports:
- containerPort: 80

Purpose:

- Exposes port 80 inside container.

Nginx listens on:

Port 80

Second Container : BusyBox

- name: busybox-container
image: busybox

Purpose:

- Creates lightweight BusyBox container.

Command Section

command: ["sh", "-c", "while true; do echo 'Hello from BusyBox'; sleep 10; done"]

Purpose:

- Runs shell command continuously.
-

Command Breakdown

sh

sh

Meaning:

- Starts shell interpreter.
-

-c Option

Faculty Definition:

-c : Tells sh to execute the following string command

Meaning:

- Execute command written after **-c**.
-

Infinite Loop

while true;

Meaning:

- Run forever continuously.
-

Echo Statement

echo 'Hello from BusyBox'

Meaning:

- Prints message.
-

Sleep Command

sleep 10

Meaning:

- Wait 10 seconds before printing again.
-

done

done

Meaning:

- Ends loop.
-

Full Logic of BusyBox Container

1. Start shell
 2. Run infinite loop
 3. Print message
 4. Sleep 10 seconds
 5. Repeat forever
-

Create Multi-Container POD

Command:

```
kubecttl apply -f pod-two-containers.yaml
```

Purpose:

- Creates POD with two containers.
-

Verify POD

Command:

```
kubecttl get pods
```

Important Faculty Note

We get single POD but 2/2 meaning 2 containers

Understanding 2/2

Example Output:

NAME	READY	STATUS
two-container-pod	2/2	Running

Meaning:

- First 2 → Running containers
- Second 2 → Total containers

So:

2/2 = Both containers are running successfully

Check Logs of BusyBox Container

Command:

```
kubectl logs two-container-pod -c busybox-container
```

Purpose:

- Shows BusyBox container logs.

Expected Output:

```
Hello from BusyBox  
Hello from BusyBox  
Hello from BusyBox
```

Repeated every 10 seconds.

Check Logs of Nginx Container

Command:

```
kubectl logs two-container-pod -c nginx-container
```

Purpose:

- Shows Nginx container logs.
-

Why Use -c Option in Logs?

Because:

- Multiple containers exist inside same POD.

Kubernetes must know:

- Which container logs you want.
-

Important Real-Time Use Cases

Sidecar Container Pattern

One container:

- Main application

Second container:

- Logging
 - Monitoring
 - Security
 - Data syncing
-

Example

Container	Purpose
Nginx	Main application
BusyBox	Helper/background task

Important Characteristics of Multi-Container POD

Shared Network

All containers:

- Share same IP address.

They communicate using:

localhost

Shared Storage

Containers can:

- Share same volume.
-

Same Lifecycle

All containers:

- Created together
- Deleted together

Because they belong to same POD.

Important Interview Question

Question:

Can a POD contain multiple containers?

Answer:

Yes.

Kubernetes POD can contain:

- One container
- Or multiple containers

Containers inside same POD share:

- Network
 - Storage
 - Lifecycle
-

Important Interview Question

Question:

Why use multi-container POD?

Answer:

Used when multiple containers need to:

- Work closely together
 - Share resources
 - Communicate frequently
-

Important Interview Question**Question:**

What does 2/2 mean in kubectl get pods?

Answer:

2/2 means:

2 containers running out of total 2 containers.

Important Interview Question**Question:**

Why use -c in kubectl logs?

Answer:

Because:

- POD contains multiple containers.

So Kubernetes needs container name to show logs.

Example:

kubectl logs two-container-pod -c busybox-container

Final Quick Revision

Multi-Container POD:

- One POD contains multiple containers.

Containers inside same POD:

- Share network
- Share storage
- Share lifecycle

Example:

- nginx-container
- busybox-container

2/2 means:

- 2 containers running successfully.

BusyBox container continuously prints:

"Hello from BusyBox"

every 10 seconds.